ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN

SUBMISSION #3

MÔN HỌC: **CÔNG NGHỆ PHẦN MỀM (CO3001)** — HK261

ĐỀ TÀI:

SMART E-MOBILITY HUB

Phân hệ: **SS5: Lập lịch Sạc Thông minh (Smart Charging Scheduler)**

Giảng viên hướng dẫn: TS. Trần Thị Ngọc Trâm

Lớp học phần: L02

Nhóm báo cáo: BEE BRAVE (Nhóm 7)

Sinh viên thực hiện: Dương Đăng Khoa

MSSV: 2411589

Revision: 06/11/2026 (Bản kỹ thuật trước nộp LMS)

Thành phố Hồ Chí Minh, Tháng 11/2026

Contents

1 Submission 3 — Dương Đăng Khoa — SS5 **2**
1.1 1. Phạm vi và đóng góp . . . 2
1.2 2. Deployment view . . . 2
1.2.1 Vị trí phân hệ SS5 . . . 2
1.2.2 Thiết bị/tiến trình, dữ liệu và giao tiếp . . . 3
1.2.3 Mô phỏng và giới hạn hiện tại . . . 3
1.3 3. Development/Implementation view . . . 3
1.3.1 Module giao diện hiện tại . . . 3
1.3.2 Core/service/adapter và phụ thuộc . . . 3
1.3.3 Ánh xạ code hiện tại / thiết kế đề xuất . . . 3
1.4 4. Class diagram cá nhân . . . 4
1.4.1 Phân biệt mức triển khai . . . 5
1.5 5. Mô tả lớp . . . 5
1.5.1 5.1. ChargingRequest . . . 5
1.5.2 5.2. ChargingRequestResult . . . 5
1.5.3 5.3. ChargingSession . . . 5
1.5.4 5.4. ChargingPort . . . 6
1.5.5 5.5. ChargingBill . . . 6
1.5.6 5.6. PortFailureResult . . . 6
1.5.7 5.7. ChargingService . . . 6
1.5.8 5.8. SmartChargingScheduler . . . 6
1.6 6. Mô tả phương thức . . . 7
1.6.1 6.1. ChargingRequest . . . 7
1.6.2 6.2. SmartChargingScheduler . . . 7
1.6.3 6.3. ChargingService — thiết kế theo hợp đồng . . . 8
1.6.4 6.4. BillingService — giao tiếp bên ngoài SS5 . . . 8
1.6.5 6.5. Các hàm billing hiện tại . . . 9
1.7 7. Truy vết Sub1-Sub2-Sub3 (revision tích hợp) . . . 9
1.8 UC_SS5_01 - Lập lịch sạc tự động theo điểm ưu tiên & Giới hạn công suất lưới . . . 9
1.9 UC_SS5_02 - Giám sát hàng chờ sạc, hoàn thành, ân hạn & Tính phí quá giờ 9
1.10 8. Test case bonus . . . 10
1.10.1 TC-SS5-B01 — Kiểm tra thứ tự ưu tiên và cấp cổng . . . 10
1.10.2 TC-SS5-B02 — Kiểm tra vượt giới hạn công suất . . . 10
1.10.3 TC-SS5-B03 — Kiểm tra hoàn thành và phí quá giờ . . . 10
1.10.4 TC-SS5-B04 — Rút xe trong thời gian ân hạn . . . 10
1.11 9. Giới hạn và quyết định thiết kế . . . 11
1.11.1 Quyết định áp dụng . . . 11
1.11.2 Giới hạn thuộc phân hệ . . . 11
1.11.3 Vấn đề đã tự xử lý / còn mở . . . 11
1.12 10. Tự kiểm tra và khai báo AI . . . 11
1.12.1 Xác nhận tự kiểm tra . . . 11
1.12.2 Khai báo sử dụng AI . . . 12
1.13 11. Bộ file revision tích hợp . . . 12
1.14 Kiến trúc chung và trạng thái phiên bản tích hợp . . . 12
1.15 Hiệu chỉnh tích hợp 06/11/2026 . . . 12

PHỤ LỤC: Danh mục Phương thức (method_catalog.md) **14**

2 Danh mục mọi phương thức SS5 - revision 06/11/2026 **14**
2.1 ChargingRequest.__post_init__ . . . 14
2.2 ChargingRequest.__repr__ . . . 14
2.3 SmartChargingScheduler.__init__ . . . 14
2.4 SmartChargingScheduler.calculate_priority . . . 14
2.5 SmartChargingScheduler.add_request . . . 14
2.6 SmartChargingScheduler.remove_request . . . 15
2.7 SmartChargingScheduler.get_sorted_queue . . . 15
2.8 SmartChargingScheduler.allocate_ports . . . 15
2.9 SmartChargingScheduler.advance_simulation_tick . . . 15
2.10 SmartChargingScheduler.unplug_vehicle . . . 15
2.11 SmartChargingScheduler.get_queue_as_dicts . . . 16
2.12 ChargingService.request_charging . . . 16
2.13 ChargingService.get_bill . . . 16
2.14 ChargingService.handle_port_failure . . . 16
2.15 BillingFunctions.settle_overstay_charging . . . 17
2.16 BillingFunctions.calculate_private_ev_bill . . . 17
2.17 BillingFunctions.record_transaction . . . 17
2.18 BillingFunctions.record_wallet_debt . . . 17
2.19 BillingService.settle_invoice . . . 18
2.20 Class descriptions bổ sung . . . 18

PHỤ LỤC: Ma trận Truy vết Yêu cầu (traceability.md) **19**

3 Truy vết SS5 - revision 06/11/2026 **19**
3.1 UC_SS5_01 - Lập lịch sạc tự động theo điểm ưu tiên & Giới hạn công suất lưới . . . 19
3.2 UC_SS5_02 - Giám sát hàng chờ sạc, hoàn thành, ân hạn & Tính phí quá giờ 19

PHỤ LỤC: Kiểm thử & Test Cases (test_cases.md) **20**

4 TEST CASE BONUS — DƯƠNG ĐĂNG KHOA — SS5 **20**
4.1 UC_SS5_01 — Lập lịch sạc tự động theo điểm ưu tiên và giới hạn công suất lưới . . . 20
4.1.1 TC-SS5-B01 — Luồng chính: Tính điểm ưu tiên và phân bổ cổng sạc 20
4.1.2 TC-SS5-B02 — Ngoại lệ: Tổng công suất vượt giới hạn hub . . . 21
4.2 UC_SS5_02 — Giám sát hàng chờ sạc, hoàn thành, ân hạn và tính phí quá giờ . . . 21
4.2.1 TC-SS5-B03 — Luồng chính: Sạc hoàn tất, hết ân hạn và phát sinh phí Overstay . . . 21
4.2.2 TC-SS5-B04 — Ngoại lệ: Xe rời cổng trong thời gian ân hạn . . . 22
4.3 Test Execution Summary . . . 23
4.4 Đối chiếu thực thi 06/11/2026 . . . 23

PHỤ LỤC: Kiến trúc Hệ thống Chung (architecture.md) **24**

5 Kiến trúc chung revision 06/11/2026 **24**
5.1 Deployment view hiện tại . . . 24
5.2 Development/Implementation view hiện tại . . . 24
5.3 Class diagram hệ thống - thiết kế mục tiêu . . . 25
5.4 Ràng buộc chất lượng và hướng mở rộng . . . 25

1 Submission 3 — Dương Đăng Khoa — SS5

> **Revision tích hợp06/11/2026:** có chỉnh sửa với Codex hỗ trợ; chủ sở hữu cần xác nhận bản mới. Truy vết/test/catalog revision này được dùng khi khác nhận định của bản bàn giao trước.

MSSV: 2411589
Phân hệ: SS5 — Lập lịch, phiên sạc và phí quá giờ
Trạng thái: DRAFT — ĐÃ HOÀN THIỆN NỘI DUNG, CHỜ KIỂM TRA KẾT QUẢ TEST VÀ XUẤT PDF
Khoảng thực hiện: 21/10/2026–30/10/2026
Hướng dẫn và phân công · Quyết định dùng chung

1.1 1. Phạm vi và đóng góp

Phân hệ SS5 phụ trách lập lịch sạc, quản lý phiên sạc và xử lý phí quá giờ tại các hub sạc.

Các use case thuộc phạm vi cá nhân:

- **UC_SS5_01** — Lập lịch sạc tự động theo điểm ưu tiên và giới hạn công suất lưới.
- **UC_SS5_02** — Giám sát hàng chờ sạc, hoàn thành phiên sạc, thời gian ân hạn và phí quá giờ.

Đóng góp thực tế của cá nhân:

- Xây dựng và hoàn thiện thuật toán tính điểm ưu tiên cho yêu cầu sạc.
- Xây dựng cơ chế xếp hàng và cấp cổng sạc theo thứ tự ưu tiên.
- Xử lý giới hạn tổng công suất sạc của hub.
- Xây dựng mô hình phiên sạc và mô phỏng tiến trình sạc.
- Xử lý thông báo khi SoC đạt ngưỡng 90% và khi sạc hoàn thành.
- Xử lý thời gian ân hạn và phí quá giờ.
- Xây dựng class diagram SS5 và mô tả trách nhiệm, thuộc tính, phương thức.
- Xây dựng nội dung truy vết, test case và tự kiểm tra phân hệ.
- Đối chiếu thiết kế SS5 với các giao tiếp dùng chung của nhóm.

Phân hệ SS5 được xây dựng theo hướng mô phỏng trong ứng dụng Python/Streamlit. Các giao tiếp ChargingService và một số kiểu dữ liệu dùng chung được mô tả theo hợp đồng kiến trúc; cần phân biệt rõ phần đã triển khai trong mã nguồn với phần thiết kế đề xuất.

1.2 2. Deployment view

Nguồn sơ đồ triển khai chung

1.2.1 Vị trí phân hệ SS5

SS5 nằm trong ứng dụng Smart E-Mobility Hub, được triển khai cùng ứng dụng Python/Streamlit. Phân hệ không phải một microservice độc lập và không có tiến trình triển khai riêng.

1.2.2 Thiết bị/tiến trình, dữ liệu và giao tiếp

- **Presentation:** giao diện Streamlit trong `src/components/smart_charging.py`.
- **Application/Core:** thuật toán lập lịch, hàng chờ, phiên sạc và xử lý phí.
- **State:** dữ liệu mô phỏng được lưu trong `st.session_state`.
- **Giao tiếp nội bộ:** các module Python gọi trực tiếp nhau trong cùng ứng dụng.
- **Giao tiếp với SS2:** theo hợp đồng ChargingService để yêu cầu sạc và truy vấn hóa đơn.
- **Giao tiếp với SS7:** theo hợp đồng xử lý sự cố cổng sạc.
- **Giao tiếp với TV1:** theo hợp đồng BillingService để chuyển các khoản phí cần thanh toán.

1.2.3 Mô phỏng và giới hạn hiện tại

Ứng dụng hiện mô phỏng quá trình sạc, cập nhật SoC, trạng thái cổng và phiên sạc. Không khẳng định hệ thống đang kết nối trực tiếp với thiết bị IoT, trạm sạc vật lý hoặc hệ thống thanh toán thực tế.

Các dữ liệu mô phỏng được lưu trong session state của Streamlit. Vì vậy, không mô tả dữ liệu SS5 hiện tại là dữ liệu được lưu bền vững trong cơ sở dữ liệu.

1.3 3. Development/Implementation view

Nguồn sơ đồ module chung

1.3.1 Module giao diện hiện tại

`src/components/smart_charging.py`

Module này chịu trách nhiệm khởi tạo và hiển thị giao diện quản lý sạc, hàng chờ, phiên sạc và quá giờ.

1.3.2 Core/service/adapter và phụ thuộc

Module	Trách nhiệm
`src/core/scheduler.py`	Định nghĩa yêu cầu sạc, phiên sạc và thuật toán lập lịch
`src/core/policies.py`	Cung cấp chính sách công suất, phí quá giờ và đơn giá điện
`src/core/billing.py`	Cung cấp các hàm xử lý giao dịch và phí
`src/core/state.py`	Cung cấp cơ chế cập nhật phiên bản state
`src/components/smart_charging.py`	Kết nối giao diện với các chức năng SS5

1.3.3 Ánh xạ code hiện tại / thiết kế đề xuất

Đã triển khai trong mã nguồn:

- `ChargingRequest`
- `ChargingSession`
- `SmartChargingScheduler`
- Các hàm xử lý phí trong `billing.py`

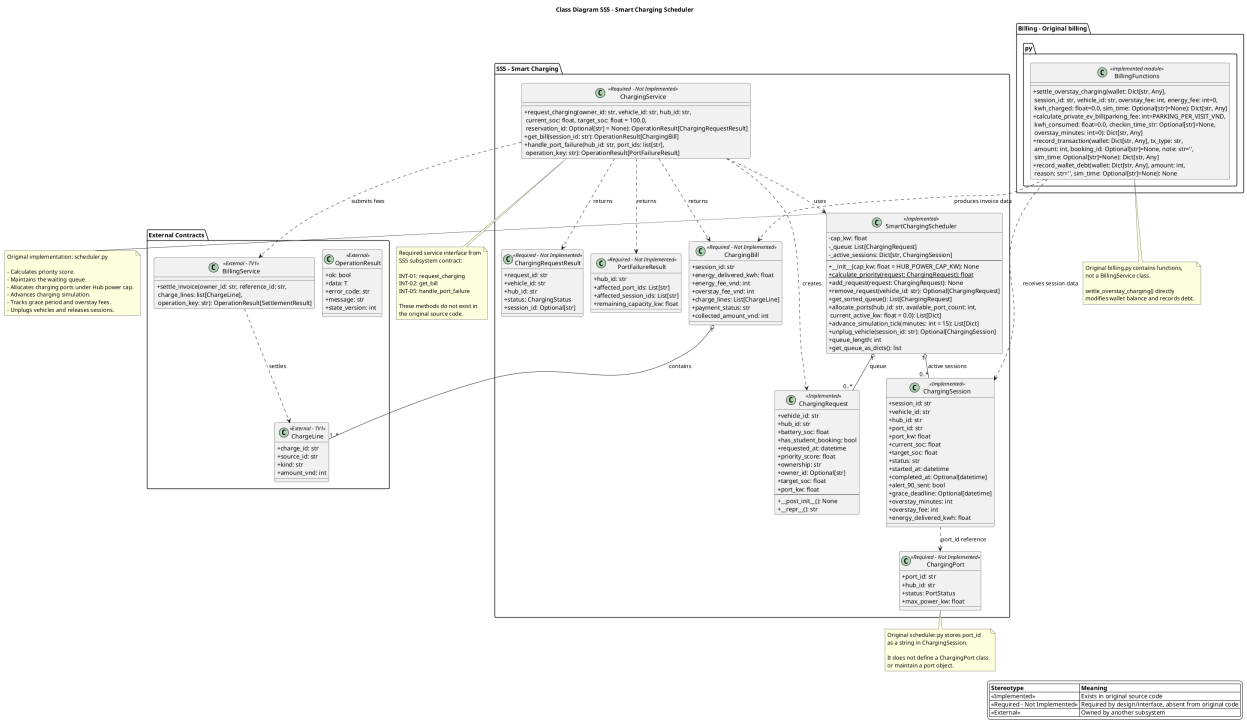


Figure 1: Class diagram SS5

- Giao diện quản lý sạc trong `smart_charging.py`

Thiết kế theo hợp đồng kiến trúc, chưa được khẳng định là đã triển khai:

- `ChargingService`
- `ChargingRequestResult`
- `ChargingPort`
- `ChargingBill`
- `PortFailureResult`
- `BillingService` của TV1

Trong code hiện tại, giao diện SS5 gọi trực tiếp hàm `settle_overstay_charging()` từ module billing. Đây là điểm cần phân biệt với kiến trúc mục tiêu, trong đó SS5 tính phí và chuyển các khoản phí sang BillingService của TV1 để xử lý thanh toán.

1.4 4. Class diagram cá nhân

Nguồn class diagram SS5

Class diagram SS5 thể hiện các lớp và kiểu dữ liệu chính của phân hệ:

- `ChargingRequest`
- `ChargingRequestResult`
- `ChargingSession`
- `ChargingPort`
- `ChargingBill`
- `PortFailureResult`
- `ChargingService`
- `SmartChargingScheduler`

Các lớp dùng chung hoặc thuộc phân hệ khác được tham chiếu theo đúng chủ sở hữu, không tạo bản sao định nghĩa.

1.4.1 Phân biệt mức triển khai

Lớp	Mức triển khai
`ChargingRequest`	Đã triển khai
`ChargingSession`	Đã triển khai
`SmartChargingScheduler`	Đã triển khai
`ChargingRequestResult`	Kiểu dữ liệu theo hợp đồng, đề xuất
`ChargingPort`	Kiểu dữ liệu theo hợp đồng, đề xuất
`ChargingBill`	Kiểu dữ liệu theo hợp đồng, đề xuất
`PortFailureResult`	Kiểu dữ liệu theo hợp đồng, đề xuất
`ChargingService`	Service theo hợp đồng, đề xuất

Trong mã hiện tại, cổng sạc được biểu diễn bằng các ID và dữ liệu trạng thái, chưa có lớp `ChargingPort` độc lập.

1.5 5. Mô tả lớp

1.5.1 5.1. ChargingRequest

- **Trách nhiệm:** biểu diễn yêu cầu sạc đang chờ cấp cổng.
- **Mức triển khai:** đã triển khai.
- **Thuộc tính:** `vehicle_id`, `hub_id`, `battery_soc`, `has_student_booking`, `requested_at`, `priority_score`, `ownership`, `owner_id`, `target_soc`, `port_kw`.
- **Quan hệ:** được `SmartChargingScheduler` tiếp nhận và sắp xếp.
- **Invariant:** SoC nằm trong phạm vi 0–100; yêu cầu phải có thông tin hub và vehicle hợp lệ.
- **Phương thức:** `__post_init__()`, `__repr__()`.

1.5.2 5.2. ChargingRequestResult

- **Trách nhiệm:** trả kết quả tiếp nhận yêu cầu sạc cho bên gọi.
- **Mức triển khai:** đề xuất theo hợp đồng.
- **Thuộc tính:** `request_id`, `vehicle_id`, `hub_id`, `status`, `session_id`.
- **Quan hệ:** kết quả của `ChargingService.request_charging()`.
- **Invariant:** yêu cầu đang chờ chưa có phiên sạc thì `session_id` không được gán.

1.5.3 5.3. ChargingSession

- **Trách nhiệm:** lưu trạng thái và dữ liệu của phiên sạc.
- **Mức triển khai:** đã triển khai.
- **Thuộc tính:** `session_id`, `vehicle_id`, `hub_id`, `port_id`, `port_kw`, `current_soc`, `target_soc`, `status`, `started_at`, `completed_at`, `alert_90_sent`, `grace_deadline`, `overstay_minutes`, `overstay_fee`, `energy_delivered_kwh`.
- **Quan hệ:** được `SmartChargingScheduler` quản lý trong danh sách phiên đang hoạt động.
- **Invariant:** một xe không được có nhiều phiên sạc hoạt động đồng thời; SoC nằm trong 0–100.
- **Phương thức:** các phương thức quản lý phiên được thực hiện bởi scheduler.

1.5.4 5.4. ChargingPort

- **Trách nhiệm:** biểu diễn cổng sạc và trạng thái khả dụng.
- **Mức triển khai:** đề xuất theo hợp đồng.
- **Thuộc tính dự kiến:** `port_id`, `hub_id`, `power_kw`, `status`, `session_id`.
- **Quan hệ:** thuộc một hub; có thể được một phiên sạc chiếm dụng.
- **Invariant:** cổng đang bận không được cấp cho phiên khác.

1.5.5 5.5. ChargingBill

- **Trách nhiệm:** biểu diễn thông tin phí điện và phí quá giờ của phiên sạc.
- **Mức triển khai:** đề xuất theo hợp đồng.
- **Thuộc tính:** `session_id`, `energy_delivered_kwh`, `energy_fee_vnd`, `overstay_fee_vnd`, `charge_lines`, `payment_info`.
- **Quan hệ:** được cung cấp cho SS2 thông qua `ChargingService.get_bill()`.
- **Invariant:** hóa đơn chỉ cung cấp dữ liệu tính phí; không tự thực hiện trừ tiền ví.

1.5.6 5.6. PortFailureResult

- **Trách nhiệm:** trả kết quả xử lý sự cố cổng sạc.
- **Mức triển khai:** đề xuất theo hợp đồng.
- **Thuộc tính:** `hub_id`, `affected_port_ids`, `affected_session_ids`, `resulting_capacity`.
- **Quan hệ:** kết quả của `ChargingService.handle_port_failure()`.
- **Invariant:** sự cố cổng sạc không tự động thay đổi trạng thái chỗ đỗ xe.

1.5.7 5.7. ChargingService

- **Trách nhiệm:** cung cấp giao tiếp SS5 cho các phân hệ khác.
- **Mức triển khai:** thiết kế theo hợp đồng, chưa khẳng định đã triển khai.
- **Phương thức chính:**
  - `request_charging(...)`
  - `get_bill(session_id)`
  - `handle_port_failure(...)`
- **Quan hệ:** SS2 gọi yêu cầu sạc và truy vấn hóa đơn; SS7 gọi xử lý sự cố cổng.

1.5.8 5.8. SmartChargingScheduler

- **Trách nhiệm:** tính điểm ưu tiên, quản lý hàng chờ, cấp cổng, giới hạn công suất và tiến trình phiên sạc.
- **Mức triển khai:** đã triển khai.
- **Thuộc tính:** `cap_kw`, `_queue`, `_active_sessions`.
- **Quan hệ:** quản lý `ChargingRequest` và `ChargingSession`.
- **Invariant:** tổng công suất các phiên sạc được cấp không vượt quá công suất giới hạn của hub.
- **Phương thức:** `calculate_priority()`, `add_request()`, `remove_request()`, `get_sorted_queue()`, `allocate_ports()`, `advance_simulation_tick()`, `unplug_vehicle()`.

1.6 6. Mô tả phương thức

1.6.1 6.1. ChargingRequest

`__post_init__()`

- Tự tính điểm ưu tiên khi yêu cầu chưa có điểm.
- Dữ liệu tác động: `priority_score`.
- Tiền điều kiện: đối tượng yêu cầu sạc đã được tạo.
- Hậu điều kiện: điểm ưu tiên được tính theo chính sách SS5.

`__repr__()`

- Trả chuỗi biểu diễn yêu cầu sạc.
- Không thay đổi trạng thái.

1.6.2 6.2. SmartChargingScheduler

`__init__(cap_kw)`

- Khởi tạo scheduler với giới hạn công suất.
- Tạo hàng chờ và danh sách phiên sạc đang hoạt động.

`calculate_priority(request)`

- Tính điểm ưu tiên theo SoC và trạng thái đặt chỗ sinh viên.
- Công thức:

```
0.5 * (100 - SoC) + 100 * I(SoC < 20) + 60 * I(has_student_booking)
+ 30 * I(SoC < 50)
```

- SoC càng thấp thì điểm ưu tiên càng cao.
- Yêu cầu SoC dưới 20 được cộng điểm ưu tiên khẩn cấp.
- Yêu cầu có booking sinh viên được cộng điểm ưu tiên.
- Yêu cầu SoC dưới 50 được cộng thêm điểm.

`add_request(request)`

- Tiếp nhận yêu cầu sạc vào hàng chờ.
- Loại yêu cầu trùng xe trước khi thêm.
- Tính lại điểm ưu tiên.

`remove_request(vehicle_id)`

- Loại yêu cầu của xe khỏi hàng chờ.
- Không tạo phiên sạc.

`get_sorted_queue()`

- Trả hàng chờ theo điểm ưu tiên giảm dần.
- Nếu điểm bằng nhau, ưu tiên thời điểm yêu cầu sớm hơn; tiếp theo là `vehicle_id` để bảo đảm thứ tự xác định.

`allocate_ports(hub_id, available_port_count, current_active_kw)`

- Xét các yêu cầu thuộc hub.
- Kiểm tra cổng còn khả dụng và tổng công suất hiện tại.
- Chỉ cấp phiên nếu không vượt giới hạn công suất.
- Yêu cầu chưa được cấp tiếp tục ở hàng chờ.

`advance_simulation_tick(minutes)`

- Cập nhật năng lượng và SoC của các phiên sạc.
- Phát sự kiện khi SoC đạt 90%.
- Khi SoC đạt 100%, bắt đầu thời gian ân hạn.
- Khi hết ân hạn mà xe vẫn chiếm cổng, chuyển sang trạng thái quá giờ và tính phí.

`unplug_vehicle(session_id)`

- Kết thúc phiên sạc và giải phóng phiên khỏi danh sách đang hoạt động.

`queue_length`

- Trả số lượng yêu cầu đang chờ.

`get_queue_as_dicts()`

- Trả hàng chờ dưới dạng danh sách dictionary phục vụ giao diện.

1.6.3 6.3. ChargingService — thiết kế theo hợp đồng

`request_charging(owner_id, vehicle_id, hub_id, current_soc, target_soc=100, reservation_id=None)`

- Bên gọi: SS2.
- Tiếp nhận yêu cầu sạc sau check-in hợp lệ.
- Kiểm tra dữ liệu, tránh tạo yêu cầu trùng cho cùng chuyến/xe.
- Kết quả: `OperationResult[ChargingRequestResult]`.

`get_bill(session_id)`

- Bên gọi: SS2/SS3.
- Trả dữ liệu đo lường và phí điện/phí quá giờ.
- Không tự trừ tiền.
- Kết quả: `OperationResult[ChargingBill]`.

`handle_port_failure(hub_id, port_ids, operation_key)`

- Bên gọi: TV7 thông qua SimulationCoordinator.
- Xử lý các cổng gặp sự cố và trả danh sách phiên bị ảnh hưởng.
- Không thay đổi chỗ đỗ xe chỉ vì cổng sạc gặp sự cố.
- Kết quả: `OperationResult[PortFailureResult]`.

1.6.4 6.4. BillingService — giao tiếp bên ngoài SS5

`settle_invoice(owner_id, reference_id, charge_lines, operation_key)`

- Chủ sở hữu: TV1.
- SS5 cung cấp các khoản phí cần thu; TV1 chịu trách nhiệm xử lý ví và thanh toán.
- Cần hỗ trợ xử lý lặp an toàn theo `operation_key` và ID khoản phí.

1.6.5 6.5. Các hàm billing hiện tại

`settle_overstay_charging(...)`

- Tính và ghi nhận phí quá giờ/điện theo luồng hiện tại.
- Trong code hiện tại có thao tác trực tiếp với wallet.
- Đây là khác biệt so với kiến trúc mục tiêu, trong đó TV1 quản lý việc thu tiền.

`calculate_private_ev_bill(...)`

- Tính phí cho phiên sạc xe điện theo dữ liệu đầu vào.
- Không được xem là một phương thức của `ChargingService` nếu chưa được đưa vào class diagram.

`record_transaction(...)` và `record_wallet_debt(...)`

- Ghi nhận giao dịch và khoản nợ ví theo luồng billing hiện tại.
- Cần đối chiếu với trách nhiệm BillingService trong kiến trúc chung.

1.7 7. Truy vết Sub1-Sub2-Sub3 (revision tích hợp)

ID US/FR lấy từ SRS, UC và đường dẫn scenario/sequence/activity kế thừa manifest Sub2. Các tên alias cũ trong trường functions của Sub2 không chứng minh code có method tương ứng; bảng này dùng tên hiện tại.

1.8 UC_SS5_01 - Lập lịch sạc tự động theo điểm ưu tiên & Giới hạn công suất lưới

- **Yêu cầu:** US-10, US-12, US-18, US-20, FR-04, FR-09, FR-11.
- **Scenario:** UC_SS5_DuongDangKhoa.md.
- **Sequence:** UC_SS5_01_DuongDangKhoa_sequence.puml.
- **Activity:** UC_SS5_01_DuongDangKhoa_activity.puml.
- **Thiết kế:** `SmartChargingScheduler.calculate_priority`, `SmartChargingScheduler.al` `ChargingService.request_charging`. Class diagram SS5, catalog cùng thư mục.
- **Luồng truy vết:** kiểm tra điều kiện/nhánh lỗi ở scenario và sequence/activity tương ứng -> phương thức phụ trách ở trên -> test prototype bên dưới; không tự đặt số bước nếu nguồn chưa có đánh số ổn định.
- **Test prototype:** `tests/test_scheduler_and_charging.py::test_priority_scores_ma` `tests/test_scheduler_and_charging.py::test_deterministic_tie_breaking`, `tests/test_scheduler_and_charging.py::test_allocation_and_power_cap`, `tests/test_submission_2_readiness_fixes.py::test_r09_repeated_charging_alloc`
- **Trạng thái:** thiết kế đã rà soát kỹ thuật; chủ sở hữu chờ xác nhận. Test core/UI hiện có không xác nhận service đề xuất đã được tích hợp đầy đủ.

1.9 UC_SS5_02 - Giám sát hàng chờ sạc, hoàn thành, ân hạn & Tính phí quá giờ

- **Yêu cầu:** US-12, US-19, US-19b, FR-05b, FR-10, FR-10b, FR-15b.
- **Scenario:** UC_SS5_DuongDangKhoa.md.
- **Sequence:** UC_SS5_02_DuongDangKhoa_sequence.puml.
- **Activity:** UC_SS5_02_DuongDangKhoa_activity.puml.

- **Thiết kế:** `SmartChargingScheduler.advance_simulation_tick`, `SmartChargingSchedul` `ChargingService.get_bill`. Class diagram SS5, catalog cùng thư mục.
- **Luồng truy vết:** kiểm tra điều kiện/nhánh lỗi ở scenario và sequence/activity tương ứng -> phương thức phụ trách ở trên -> test prototype bên dưới; không tự đặt số bước nếu nguồn chưa có đánh số ổn định.
- **Test prototype:** `tests/test_scheduler_and_charging.py::test_simulation_tick_an` `tests/test_submission_2_acceptance.py::test_charging_completion_notification` `tests/test_submission_2_readiness_fixes.py::test_r11_overstay_charging_settl`
- **Trạng thái:** thiết kế đã rà soát kỹ thuật; chủ sở hữu chờ xác nhận. Test core/UI hiện có không xác nhận service đề xuất đã được tích hợp đầy đủ.

1.10 8. Test case bonus

Các test case bonus được xây dựng để kiểm tra thuật toán ưu tiên, thứ tự cấp cổng, giới hạn công suất và tiến trình phiên sạc.

1.10.1 TC-SS5-B01 — Kiểm tra thứ tự ưu tiên và cấp cổng

- **UC:** UC_SS5_01
- **Đầu vào:** giới hạn công suất 20 kW; ba yêu cầu sạc có SoC lần lượt 5%, 30% có booking, 30% không booking; công suất mỗi yêu cầu 7.4 kW.
- **Kết quả mong đợi:** điểm ưu tiên lần lượt 177.5, 125.0, 65.0; cấp hai yêu cầu đầu tiên; yêu cầu còn lại tiếp tục chờ.
- **Trạng thái:** Đã thiết kế; chưa xác nhận PASS bằng kết quả chạy thực tế.

1.10.2 TC-SS5-B02 — Kiểm tra vượt giới hạn công suất

- **UC:** UC_SS5_01
- **Đầu vào:** giới hạn 20 kW; hai yêu cầu sạc, mỗi yêu cầu 11 kW.
- **Kết quả mong đợi:** chỉ cấp một phiên sạc; yêu cầu thứ hai tiếp tục chờ do không đủ công suất.
- **Trạng thái:** Đã thiết kế; chưa xác nhận PASS bằng kết quả chạy thực tế.

1.10.3 TC-SS5-B03 — Kiểm tra hoàn thành và phí quá giờ

- **UC:** UC_SS5_02
- **Đầu vào:** phiên sạc SoC 98%, công suất 7.4 kW; mô phỏng 15 phút, tiếp tục 15 phút, rồi thêm 5 phút.
- **Kết quả mong đợi:** chuyển sang grace period khi đạt 100%; hết 15 phút ân hạn chuyển sang quá giờ; phí một block là 10.000 VND; thao tác rút xe giải phóng phiên.
- **Trạng thái:** Đã thiết kế; chưa xác nhận PASS bằng kết quả chạy thực tế.

1.10.4 TC-SS5-B04 — Rút xe trong thời gian ân hạn

- **UC:** UC_SS5_02
- **Đầu vào:** phiên sạc SoC 98%, công suất 7.4 kW; mô phỏng 15 phút để đạt trạng thái ân hạn; rút xe trước khi hết ân hạn.
- **Kết quả mong đợi:** phiên được kết thúc; cổng được giải phóng; không phát sinh phí quá giờ.

- **Trạng thái:** Test case đề xuất; cần kiểm tra lại với yêu cầu và mã nguồn trước khi xác nhận.

Test source: `tests/test_scheduler_and_charging.py`
Lệnh chạy:

```
pytest -v tests/test_scheduler_and_charging.py
```

Chỉ ghi trạng thái PASS khi có kết quả thực thi thành công. Không suy ra PASS từ việc test case đã được viết.

1.11 9. Giới hạn và quyết định thiết kế

1.11.1 Quyết định áp dụng

- SS5 quản lý hàng chờ, ưu tiên, cấp cổng, công suất, phiên sạc và phí điện/phí quá giờ.
- SS2 chịu trách nhiệm luồng booking/check-in; SS5 tiếp nhận yêu cầu sạc theo hợp đồng.
- SS7 điều phối mô phỏng và sự cố cổng theo hợp đồng.
- TV1 sở hữu trách nhiệm xử lý thanh toán ví.
- SS5 không tạo bản sao các lớp dùng chung và không tự định nghĩa lại Hub.
- Các đơn vị sử dụng thống nhất: SoC %, công suất kW, năng lượng kWh, tiền VND.
- Thời gian ân hạn 15 phút và phí quá giờ theo block 15 phút, 10.000 VND/block được giữ độc lập với hold booking 30 phút của SS2.

1.11.2 Giới hạn thuộc phân hệ

- Mô phỏng sạc hiện sử dụng dữ liệu trong session state của Streamlit.
- Không xác nhận có tích hợp IoT thật hoặc thiết bị sạc vật lý.
- Không xác nhận có thanh toán thật.
- Không xác nhận có cơ sở dữ liệu bền vững cho phiên sạc.
- Các giao tiếp ChargingService và xử lý sự cố cổng được mô tả theo hợp đồng kiến trúc; cần đánh dấu rõ phần nào chưa có trong code hiện tại.

1.11.3 Vấn đề đã tự xử lý / còn mở

- Đã xác định khác biệt giữa luồng billing hiện tại và kiến trúc mục tiêu TV1.
- Đã phân biệt lớp đã triển khai với lớp đề xuất.
- Cần đối chiếu giao tiếp SS2/SS7 và BillingService theo phiên bản hợp đồng chung đã chốt.
- Cần lưu kết quả chạy test thực tế và kiểm tra PDF trước khi bàn giao.

1.12 10. Tự kiểm tra và khai báo AI

1.12.1 Xác nhận tự kiểm tra

CHƯA XÁC NHẬN.
Tác giả chỉ cập nhật trạng thái xác nhận sau khi tự kiểm tra class diagram, method descriptions, traceability, code mapping, test results và PDF đầu ra.

1.12.2 Khai báo sử dụng AI

AI được sử dụng để hỗ trợ:

- Tổ chức và hoàn thiện cấu trúc báo cáo theo mẫu Submission 3.
- Hỗ trợ viết mô tả lớp, phương thức, thuật toán và truy vết.
- Hỗ trợ rà soát sự nhất quán giữa code hiện tại và thiết kế đề xuất.
- Hỗ trợ xây dựng test case và checklist tự kiểm tra.

Tác giả chịu trách nhiệm kiểm tra tính chính xác, đối chiếu với mã nguồn và tài liệu nhóm, chạy test, xác nhận kết quả và hoàn thiện bản nộp cuối cùng.

1.13 11. Bộ file revision tích hợp

PDF chuẩn được build vào output/submission_3/ từ nguồn Markdown/PlantUML cùng revision, kèm catalog, truy vết, test, self-check, AI và kiến trúc chung. Chủ sở hữu cần mở lại, xác nhận và nộp theo quy định LMS đã kiểm chứng.

1.14 Kiến trúc chung và trạng thái phiên bản tích hợp

Revision 06/11/2026. Một ứng dụng Python/Streamlit: trình duyệt -> app.py -> components/core -> dữ liệu seed và session ở tiến trình server. Không có database bắt buộc, SSO/IoT/payment thật hay các microservice đang vận hành. Session thuộc từng kết nối Streamlit; reset nạp lại seed; rerun làm mới giao diện, không tạo telemetry nền. Chạy từ repository: `streamlit run src/app.py`; host/cổng phụ thuộc cấu hình, không cố định trong dự án.

1.15 Hiệu chỉnh tích hợp 06/11/2026

Các chữ ký ChargingService/BillingFunctions được bổ sung kiểu/tham số mặc định đúng code hoặc hợp đồng. Ảnh class_diagram thiếu đuôi được thay bằng PNG/SVG chuẩn từ đúng nguồn. Catalog bổ sung điều kiện, tác động và lỗi mọi phương thức; test mới thiết kế vẫn DESIGNED. Không khẳng định ChargingService đã được tích hợp hoặc billing đã có dedup charge_id. Nguồn DOCX/PDF cũ lưu trong lịch sử Git, không dùng thay bản build mới.

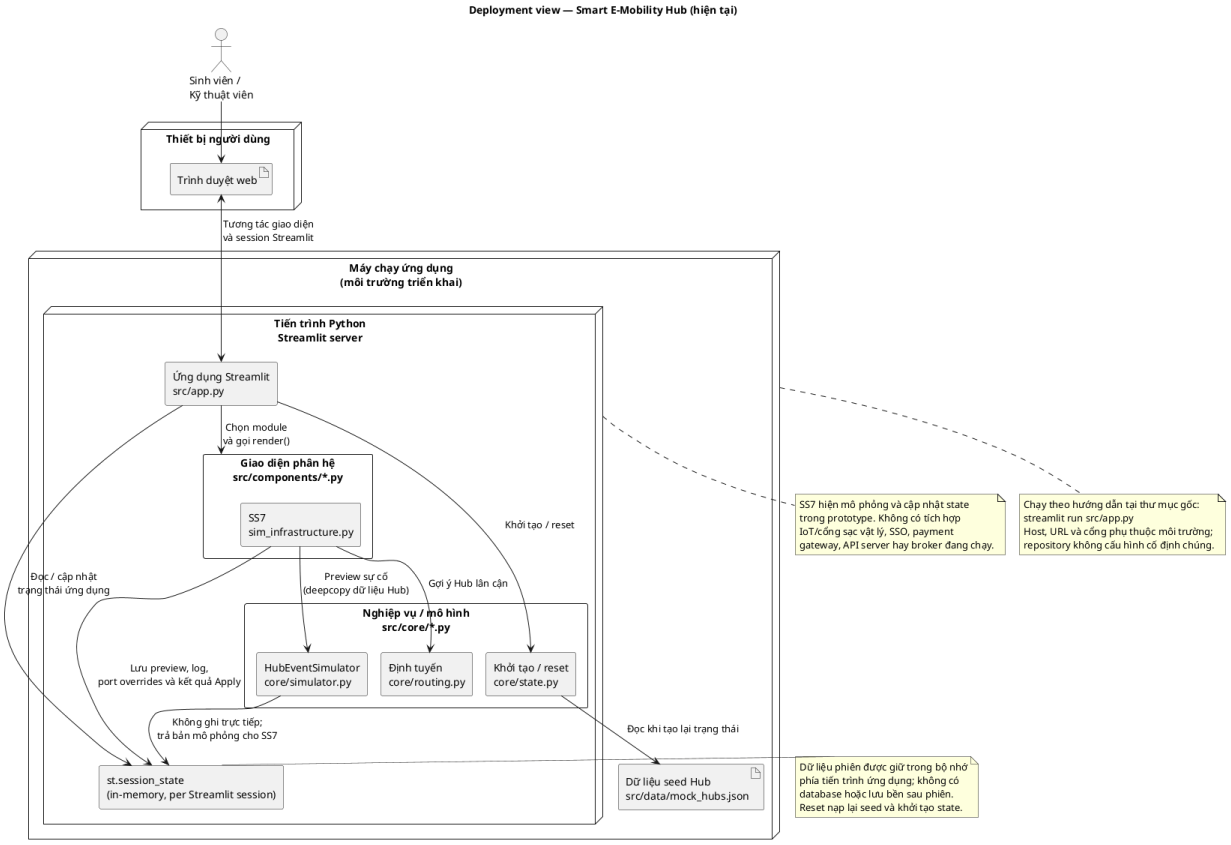


Figure 2: Deployment view chung

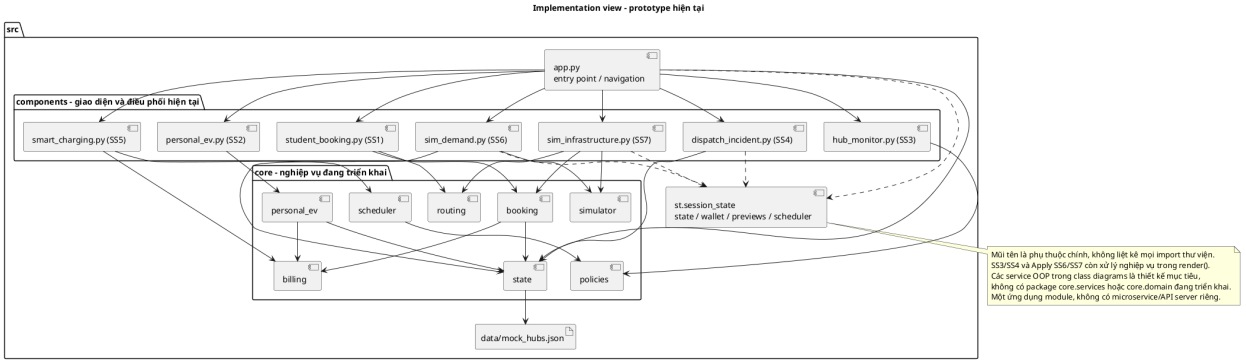


Figure 3: Development/Implementation view chung

PHỤ LỤC: Danh mục Phương thức (method_catalog.md)

2 Danh mục mọi phương thức SS5 - revision 06/11/2026

Chữ ký dưới đây khớp class diagram. Mô tả bổ sung cho bản bàn giao; kiểu dữ liệu/đơn vị và class descriptions đọc cùng REPORT_SUB3.md. Không suy ra service đề xuất đã tồn tại từ test của hàm core.

2.1 ChargingRequest.__post_init__

Chữ ký: `__post_init__() -> None`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: ChargingRequest đã có. Sau khởi tạo dataclass, nếu priority_score==0 thì tính từ calculate_priority; score khác0 được giữ. Cần battery_soc hữu hạn0–100 từ caller; core chưa validation đầy đủ. Không thêm queue hoặc thu phí.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.2 ChargingRequest.__repr__

Chữ ký: `__repr__() -> str`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: ChargingRequest đã có. Tạo chuỗi vehicle/SoC/priority để debug, không cập nhật request/queue. Gọi lặp cho cùng dữ liệu trả cùng chuỗi.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.3 SmartChargingScheduler.__init__

Chữ ký: `__init__(cap_kw: float = HUB_POWER_CAP_KW) -> None`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Scheduler đã có. Nhận cap_kw mặc định150 kW, tạo queue và active_sessions rỗng. Thiết kế ràng buộc cap_kw>0/hữu hạn; khởi tạo không đặt cổng/thu phí.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.4 SmartChargingScheduler.calculate_priority

Chữ ký: `calculate_priority(request: ChargingRequest) -> float`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Static đã có. Score=0.5*(100-SoC)+100*I(SoC<20)+60*I(has_booking)+30*I(SoC<50). Trả float; ví dụ SoC5 không booking177.5, SoC30 không booking65, có booking125. Thuần đọc; chưa có thành phần giá điện hoặc thời gian chờ tăng điểm theo SRS, requested_at chỉ dùng phá hòa.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.5 SmartChargingScheduler.add_request

Chữ ký: `add_request(request: ChargingRequest) -> None`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Đã có: bỏ request cũ cùng vehicle_id khỏi queue, tính lại score rồi append request mới. Không tự allocate hoặc kiểm tra

trùng xe trong active_sessions; yêu cầu active-dedup do service thiết kế kiểm tra. Không đổi wallet/cổng.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.6 SmartChargingScheduler.remove_request

Chữ ký: `remove_request(vehicle_id: str) -> Optional[ChargingRequest]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Đã có: tìm vehicle_id trong queue, xóa và trả ChargingRequest hoặc None nếu không có. Gọi lặp trả None và không tác động session đang sạc.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.7 SmartChargingScheduler.get_sorted_queue

Chữ ký: `get_sorted_queue() -> List[ChargingRequest]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Đã có: trả bản sắp xếp theo (-priority_score,requested_at,vehicle_id), không sửa thứ tự lưu của queue hay allocate. Bảo đảm tie deterministic khi requested_at/ID đã cố định.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.8 SmartChargingScheduler.allocate_ports

Chữ ký: `allocate_ports(hub_id: str, available_port_count: int, current_active_kw: float = 0.0) -> List[Dict]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Đã có: theo Hub và số cổng available, ưu tiên queue, kiểm tra active power đang có và current_active_kw trước thêm port_kw; trả danh sách allocation, xóa request được nhận và tạo session CHARGING. Không vượt cap khi gọi liên tiếp; không tự giảm charging_ports của Hub ngoài scheduler, UI/service chịu đồng bộ. Caller loại cổng offline. Không đảm bảo port ID vật lý hoặc giao dịch SS2/SS7.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.9 SmartChargingScheduler.advance_simulation_tick

Chữ ký: `advance_simulation_tick(minutes: int = 15) -> List[Dict]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Đã có: tăng mức pin/energy theo minutes, phát ngưỡng90%, đạt target ghi completed và GRACE_PERIOD. Sau15 phút grace, mỗi khối15 phút bắt đầu tính10.000 VNĐ; cập nhật session, trả sự kiện. Mỗi lời gọi là một bước thời gian mới, không idempotent; retry phải do service/key kiểm soát. Không tự thu ví; core dùng giờ session thực khi tạo, cần adapter đồng hồ mô phỏng.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.10 SmartChargingScheduler.unplug_vehicle

Chữ ký: `unplug_vehicle(session_id: str) -> Optional[ChargingSession]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Đã có: pop session_id khỏi active_sessions, trả ChargingSession hoặc None. Lặp không tháo phiên hai lần. UI/service trả port/count và chuyển trạng thái xe, đối soát phí; hàm scheduler không tự thay Hub/wallet.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.11 SmartChargingScheduler.get_queue_as_dicts

Chữ ký: `get_queue_as_dicts() -> list`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Đã có: trả danh sách dictionary biểu diễn queue cho UI, không tạo request hay quyết toán. Không coi returned list là dữ liệu giao dịch có khóa hoặc snapshot toàn hệ thống.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.12 ChargingService.request_charging

Chữ ký: `request_charging(owner_id: str, vehicle_id: str, hub_id: str, current_soc: float, target_soc: float = 100.0, reservation_id: Optional[str] = None) -> OperationResult[ChargingRequestResult]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: INT-01 (thiết kế TV5). Xe/owner/Hub hợp lệ, check-in đã thành công, SoC0–100 và target>=current. Tạo request có ID ổn định, chưa có session/cổng khi còn chờ; thêm queue, cấp theo power/cổng thực khi có khả năng. Không cấp trùng cùng lượt/xe; NOT_FOUND/FORBIDDEN/INVALID_STATE/INVALID_INPUT không commit. Prototype SS2 giảm cổng tổng hợp trực tiếp, chưa gọi service này.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.13 ChargingService.get_bill

Chữ ký: `get_bill(session_id: str) -> OperationResult[ChargingBill]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: INT-02 (thiết kế TV5). Tìm session_id và đối soát energy_delivered_kwh, giá/phí, charge_id và khoản đã thu từ ledger. Trả ChargingBill chỉ đọc; gọi lặp không trừ ví hoặc tăng phí giả. Phiên không có trả NOT_FOUND. SS2 lấy dòng phí còn phải thu, không tính lại giá điện/quá giờ độc lập.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.14 ChargingService.handle_port_failure

Chữ ký: `handle_port_failure(hub_id: str, port_ids: list[str], operation_key: str) -> OperationResult[PortFailureResult]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: INT-05 (thiết kế TV5), chỉ trong coordinator Apply SS7. Kiểm tra Hub, port_ids duy nhất thuộc Hub và operation_key; ngắt các phiên liên quan, đưa về chờ hợp lệ, cập nhật port offline và công suất, trả PortFailureResult. Không thay capacity đỗ. Stage cùng preview, lỗi rollback toàn bộ. Lặp key/cùng nội dung không ngắt lần hai; CONFLICTING_OPERATION/NOT_FOUND/INVALID_STATE. Prototype chưa có adapter này và cổng chỉ là số đếm ở simulator.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.15 BillingFunctions.settle_overstay_charging

Chữ ký: `settle_overstay_charging(wallet: Dict[str, Any], session_id: str, vehicle_id: str, overstay_fee: int, energy_fee: int=0, kwh_charged: float=0.0, sim_time: Optional[str]=None) -> Dict[str, Any]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Hàm module billing hiện có. Nhận phí overstay/energy đã tính và session_id/vehicle_id; thu tối đa số dư, ghi nợ còn thiếu, append invoice/ledger. Không tự tái tính tariff. Không có operation_key/charge_id dedup toàn cục; caller chỉ gọi một lần theo vòng đời unplug. Phí tiền VNĐ nguyên không âm là điều kiện caller phải bảo vệ.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.16 BillingFunctions.calculate_private_ev_bill

Chữ ký: `calculate_private_ev_bill(parking_fee: int=PARKING_PER_VISIT_VND, kwh_consumed: float=0.0, checkin_time_str: Optional[str]=None, overstay_minutes: int=0) -> Dict[str, Any]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Hàm module billing hiện có, thuần tính toán phí đỗ/kWh theo khung giờ và phí overstay. Trả dictionary phân rã phí, không đổi ví/Hub. parking_fee mặc định5.000 VNĐ/lượt, tariff7.000 cao điểm/3.000 thấp điểm; không mô hình hourly parking hay tích phân biểu giá theo từng tick.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.17 BillingFunctions.record_transaction

Chữ ký: `record_transaction(wallet: Dict[str, Any], tx_type: str, amount: int, booking_id: Optional[str]=None, note: str='', sim_time: Optional[str]=None) -> Dict[str, Any]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Hàm module billing hiện có: append ledger entry mới với tx_id, time/type/amount và balance_after/held_after/debt_after, booking_id/note. Caller đã thực hiện thay đổi số dư; hàm không tự giữ/hoàn/thu và không kiểm tra dedup. Gọi hai lần tạo hai entry; đây không là một API idempotent.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.18 BillingFunctions.record_wallet_debt

Chữ ký: `record_wallet_debt(wallet: Dict[str, Any], amount: int, reason: str='', sim_time: Optional[str]=None) -> None`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Hàm module billing hiện có: cộng khoản thiếu vào debt và ghi ledger theo reason/sim_time; amount<=0 không phát sinh nợ. Gọi lại cùng amount vẫn có thể cộng thêm; caller kiểm soát vòng đời. Không coi ghi nợ là doanh thu thực thu.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.19 BillingService.settle_invoice

Chữ ký: `settle_invoice(owner_id: str, reference_id: str, charge_lines: list[ChargeLine], operation_key: str) -> OperationResult[SettlementResult]`.

Điều kiện, hành vi, kết quả, lỗi và gọi lặp: Chỉ thu charge_id chưa thu, cùng ID khác tiền trả CONFLICTING_CHARGE. Đối trừ/hoàn cọc đúng reference rồi trừ số dư tối đa phần có thể thu và ghi nợ phần thiếu; paid/refunded/debt lấy từ ledger. Gọi lặp cùng key trả kết quả cũ, không thu lại phí SS5 ở SS2. Prototype có nhiều hàm thu trực tiếp, chưa có BillingService hay dedup charge_id. Thiết kế phải stage tất cả thay đổi trước commit.

Truy vết: các UC/method liên quan được dẫn trong traceability.md; test và trạng thái thực thi ở test_cases.md.

2.20 Class descriptions bổ sung

`BillingFunctions` là nhóm hàm ở core/billing.py, không phải class instance; mọi phương thức được vẽ đã mô tả ở trên. `BillingService`, `ChargeLine`, `OperationResult` là hợp đồng thuộc TV1, không định nghĩa lại thành lớp SS5. DTO ChargingRequestResult/ChargingBill/PortFailureResult và ChargingPort thuộc TV5, không có phương thức ngoài các thuộc tính; số liệu kWh/kW/VNĐ và quan hệ được ghi trong báo cáo. `queue_length` là property đọc int, không phải method bị bỏ mô tả; đọc len(_queue), không thay đổi dữ liệu.

PHỤ LỤC: Ma trận Truy vết Yêu cầu (traceability.md)

3 Truy vết SS5 - revision 06/11/2026

ID US/FR lấy từ SRS, UC và đường dẫn scenario/sequence/activity kế thừa manifest Sub2. Các tên alias cũ trong trường functions của Sub2 không chứng minh code có method tương ứng; bảng này dùng tên hiện tại.

3.1 UC_SS5_01 - Lập lịch sạc tự động theo điểm ưu tiên & Giới hạn công suất lưới

- **Yêu cầu:** US-10, US-12, US-18, US-20, FR-04, FR-09, FR-11.
- **Scenario:** UC_SS5_DuongDangKhoa.md.
- **Sequence:** UC_SS5_01_DuongDangKhoa_sequence.puml.
- **Activity:** UC_SS5_01_DuongDangKhoa_activity.puml.
- **Thiết kế:** `SmartChargingScheduler.calculate_priority`, `SmartChargingScheduler.al` `ChargingService.request_charging`. Class diagram SS5, catalog cùng thư mục.
- **Luồng truy vết:** kiểm tra điều kiện/nhánh lỗi ở scenario và sequence/activity tương ứng -> phương thức phụ trách ở trên -> test prototype bên dưới; không tự đặt số bước nếu nguồn chưa có đánh số ổn định.
- **Test prototype:** `tests/test_scheduler_and_charging.py::test_priority_scores_ma` `tests/test_scheduler_and_charging.py::test_deterministic_tie_breaking`, `tests/test_scheduler_and_charging.py::test_allocation_and_power_cap`, `tests/test_submission_2_readiness_fixes.py::test_r09_repeated_charging_alloc`
- **Trạng thái:** thiết kế đã rà soát kỹ thuật; chủ sở hữu chờ xác nhận. Test core/UI hiện có không xác nhận service đề xuất đã được tích hợp đầy đủ.

3.2 UC_SS5_02 - Giám sát hàng chờ sạc, hoàn thành, ân hạn & Tính phí quá giờ

- **Yêu cầu:** US-12, US-19, US-19b, FR-05b, FR-10, FR-10b, FR-15b.
- **Scenario:** UC_SS5_DuongDangKhoa.md.
- **Sequence:** UC_SS5_02_DuongDangKhoa_sequence.puml.
- **Activity:** UC_SS5_02_DuongDangKhoa_activity.puml.
- **Thiết kế:** `SmartChargingScheduler.advance_simulation_tick`, `SmartChargingSchedul` `ChargingService.get_bill`. Class diagram SS5, catalog cùng thư mục.
- **Luồng truy vết:** kiểm tra điều kiện/nhánh lỗi ở scenario và sequence/activity tương ứng -> phương thức phụ trách ở trên -> test prototype bên dưới; không tự đặt số bước nếu nguồn chưa có đánh số ổn định.
- **Test prototype:** `tests/test_scheduler_and_charging.py::test_simulation_tick_an` `tests/test_submission_2_acceptance.py::test_charging_completion_notification` `tests/test_submission_2_readiness_fixes.py::test_r11_overstay_charging_settl`
- **Trạng thái:** thiết kế đã rà soát kỹ thuật; chủ sở hữu chờ xác nhận. Test core/UI hiện có không xác nhận service đề xuất đã được tích hợp đầy đủ.

PHỤ LỤC: Kiểm thử & Test Cases (test_cases.md)

4 TEST CASE BONUS — DƯƠNG ĐĂNG KHOA — SS5

Trạng thái: Đã thiết kế test case (DESIGNED); chưa chạy kiểm thử Sub3 (NOT_RUN). Không có kết quả PASS mặc định.

4.1 UC_SS5_01 — Lập lịch sạc tự động theo điểm ưu tiên và giới hạn công suất lưới

4.1.1 TC-SS5-B01 — Luồng chính: Tính điểm ưu tiên và phân bổ cổng sạc

- **Test ID:** TC-SS5-B01
- **Method ID:** `SmartChargingScheduler.calculate_priority()`, `SmartChargingScheduler`
  `SmartChargingScheduler.allocate_ports()`
- **Tiền điều kiện:**
  - Scheduler được khởi tạo với giới hạn công suất 20 kW.
  - Hub H1 có 3 cổng sạc khả dụng.
  - Không có phiên sạc nào đang hoạt động.
- **Dữ liệu đầu vào:**

Vehicle ID	SoC	Student Booking	Port Power
EV-01	5%	No	7.4 kW
EV-02	30%	Yes	7.4 kW
EV-03	30%	No	7.4 kW

- **Các bước thực hiện:**
  1. Khởi tạo `SmartChargingScheduler(cap_kw=20.0)`.
  2. Tạo ba `ChargingRequest` tương ứng với dữ liệu đầu vào.
  3. Gọi `add_request()` cho từng request.
  4. Gọi `get_sorted_queue()` để kiểm tra thứ tự ưu tiên.
  5. Gọi `allocate_ports(hub_id="H1", available_port_count=3)`.
  6. Kiểm tra số phiên sạc được tạo và số request còn lại trong queue.
- **Kết quả mong đợi:**
  - EV-01 có Priority Score = 177.5.
  - EV-02 có Priority Score = 125.0.
  - EV-03 có Priority Score = 65.0.
  - Thứ tự ưu tiên: EV-01 → EV-02 → EV-03.
  - Chỉ hai phương tiện đầu tiên được phân bổ cổng vì tổng công suất là 14.8 kW; nếu phân bổ thêm EV-03 thì tổng công suất là 22.2 kW, vượt giới hạn 20 kW.
  - Hai phiên sạc được tạo; một request còn lại trong queue.
- **Kết quả thực tế / bằng chứng:** CHƯA CHẠY.
- **Trạng thái:** DESIGNED / NOT_RUN.

4.1.2 TC-SS5-B02 — Ngoại lệ: Tổng công suất vượt giới hạn hub

- **Test ID:** TC-SS5-B02
- **Method ID:** `SmartChargingScheduler.add_request()`, `SmartChargingScheduler.alloca`
- **Tiền điều kiện:**
  - Scheduler được khởi tạo với giới hạn công suất 20 kW.
  - Hub H1 có 3 cổng khả dụng.
  - Không có phiên sạc đang hoạt động.
- **Dữ liệu đầu vào:**

Vehicle ID	SoC	Port Power
EV-04	10%	11 kW
EV-05	15%	11 kW

- **Các bước thực hiện:**
  1. Khởi tạo `SmartChargingScheduler(cap_kw=20.0)`.
  2. Tạo hai `ChargingRequest` cho EV-04 và EV-05.
  3. Thêm hai request vào hàng đợi.
  4. Gọi `allocate_ports(hub_id="H1", available_port_count=3)`.
  5. Kiểm tra kết quả phân bổ và hàng đợi sau khi xử lý.
- **Kết quả mong đợi:**
  - EV-04 được ưu tiên phân bổ trước.
  - Công suất sau khi phân bổ EV-04 là 11 kW.
  - Không thể phân bổ EV-05 vì tổng công suất dự kiến là 22 kW, vượt giới hạn 20 kW.
  - Chỉ một phiên sạc được tạo.
  - EV-05 tiếp tục ở trạng thái chờ trong queue.
  - Không có phiên sạc nào làm tổng công suất vượt 20 kW.
- **Kết quả thực tế / bằng chứng:** CHƯA CHẠY.
- **Trạng thái:** DESIGNED / NOT_RUN.

4.2 UC_SS5_02 — Giám sát hàng chờ sạc, hoàn thành, ân hạn và tính phí quá giờ

4.2.1 TC-SS5-B03 — Luồng chính: Sạc hoàn tất, hết ân hạn và phát sinh phí Overstay

- **Test ID:** TC-SS5-B03
- **Method ID:** `SmartChargingScheduler.advance_simulation_tick()`, `SmartChargingSche`
- **Tiền điều kiện:**
  - Scheduler được khởi tạo với giới hạn công suất 150 kW.
  - Hub H1 có ít nhất một cổng sạc khả dụng.
  - Không có phiên sạc đang hoạt động.
- **Dữ liệu đầu vào:**
  - Vehicle ID: EV-06.

  - Initial SoC: 98%.
  - Port Power: 7.4 kW.
  - Target SoC: 100%.
  - Grace Period: 15 phút.
  - Overstay Rate: 10,000 VND / 15 phút.

- **Các bước thực hiện:**

  1. Khởi tạo `SmartChargingScheduler(cap_kw=150.0)`.
  2. Tạo request cho EV-06 với SoC 98% và công suất cổng 7.4 kW.
  3. Thêm request vào queue và gọi `allocate_ports()`.
  4. Lấy phiên sạc vừa tạo từ `_active_sessions`.
  5. Gọi `advance_simulation_tick(minutes=15)` để mô phỏng quá trình sạc.
  6. Kiểm tra trạng thái phiên sạc sau khi đạt SoC 100%.
  7. Gọi `advance_simulation_tick(minutes=15)` để mô phỏng thời gian ân hạn.
  8. Gọi `advance_simulation_tick(minutes=5)` để mô phỏng xe tiếp tục chiếm cổng sau thời gian ân hạn.
  9. Kiểm tra trạng thái Overstay và phí phát sinh.
  10. Gọi `unplug_vehicle(session_id)` để giải phóng phiên sạc.

- **Kết quả mong đợi:**

  - Phiên sạc chuyển sang `GRACE_PERIOD` sau khi đạt SoC 100%.
  - Trong 15 phút ân hạn, phiên vẫn ở trạng thái `GRACE_PERIOD`.
  - Sau khi vượt quá thời gian ân hạn, phiên chuyển sang `OVERSTAY`.
  - Phí Overstay được tính là 10,000 VND.
  - Gọi `unplug_vehicle()` trả về phiên sạc đã kết thúc.
  - Phiên sạc được loại khỏi `_active_sessions`.

- **Kết quả thực tế / bằng chứng:** CHƯA CHẠY.
- **Trạng thái:** DESIGNED / NOT_RUN.

4.2.2 TC-SS5-B04 — Ngoại lệ: Xe rời cổng trong thời gian ân hạn

- **Test ID:** TC-SS5-B04
- **Method ID:** `SmartChargingScheduler.advance_simulation_tick()`, `SmartChargingSche`
- **Tiền điều kiện:**

  - Scheduler được khởi tạo với giới hạn công suất 150 kW.
  - Hub H1 có ít nhất một cổng sạc khả dụng.
  - Không có phiên sạc đang hoạt động.

- **Dữ liệu đầu vào:**

  - Vehicle ID: EV-07.
  - Initial SoC: 98%.
  - Port Power: 7.4 kW.
  - Target SoC: 100%.
  - Grace Period: 15 phút.

- **Các bước thực hiện:**

  1. Khởi tạo `SmartChargingScheduler(cap_kw=150.0)`.
  2. Tạo request cho EV-07 với SoC 98%.
  3. Thêm request vào queue và phân bổ cổng sạc.
  4. Lấy phiên sạc từ `_active_sessions`.

5. Gọi `advance_simulation_tick(minutes=15)` để đưa phiên sạc đến trạng thái hoàn tất.
6. Kiểm tra phiên sạc chuyển sang `GRACE_PERIOD`.
7. Trong thời gian ân hạn, gọi `unplug_vehicle(session_id)`.
8. Kiểm tra phiên sạc đã được giải phóng và không phát sinh phí Overstay.

- **Kết quả mong đợi:**
  - Phiên sạc đạt trạng thái `GRACE_PERIOD`.
  - Khi xe rời cổng trong thời gian ân hạn, phiên sạc được kết thúc.
  - Phiên sạc được loại khỏi `_active_sessions`.
  - Không phát sinh phí Overstay.
  - Cổng sạc có thể được sử dụng cho request tiếp theo.
- **Kết quả thực tế / bằng chứng:** CHƯA CHẠY.
- **Trạng thái:** DESIGNED / NOT_RUN.

4.3 Test Execution Summary

Test ID	Use Case	Test Type	Status
TC-SS5-B01	UC_SS5_01	Main Flow	DESIGNED / NOT_RUN
TC-SS5-B02	UC_SS5_01	Exception Flow	DESIGNED / NOT_RUN
TC-SS5-B03	UC_SS5_02	Main Flow	DESIGNED / NOT_RUN
TC-SS5-B04	UC_SS5_02	Exception Flow	DESIGNED / NOT_RUN

Test File: `tests/test_scheduler_and_charging.py` (unit tests hiện có); bốn bonus test case ở trên là thiết kế bổ sung, chưa được xác nhận đã viết thành test function.

Execution Command:

```
pytest -v tests/test_scheduler_and_charging.py
```

Important: Chỉ cập nhật trạng thái thành PASS hoặc FAIL sau khi thực thi test và lưu lại bằng chứng tương ứng.

4.4 Đối chiếu thực thi 06/11/2026

Bộ49 test hiện có đã chạy PASS (log baseline chung). Các tên test dẫn trong traceability.md được đối chiếu với nguồn. Nhãn PASS của bản bàn giao chỉ có hiệu lực trong phạm vi các assert thực sự có trong test đó; các điều kiện mở rộng/operation_key/service đề xuất vẫn DESIGNED. Không dùng test của core để xác nhận service chưa tồn tại.

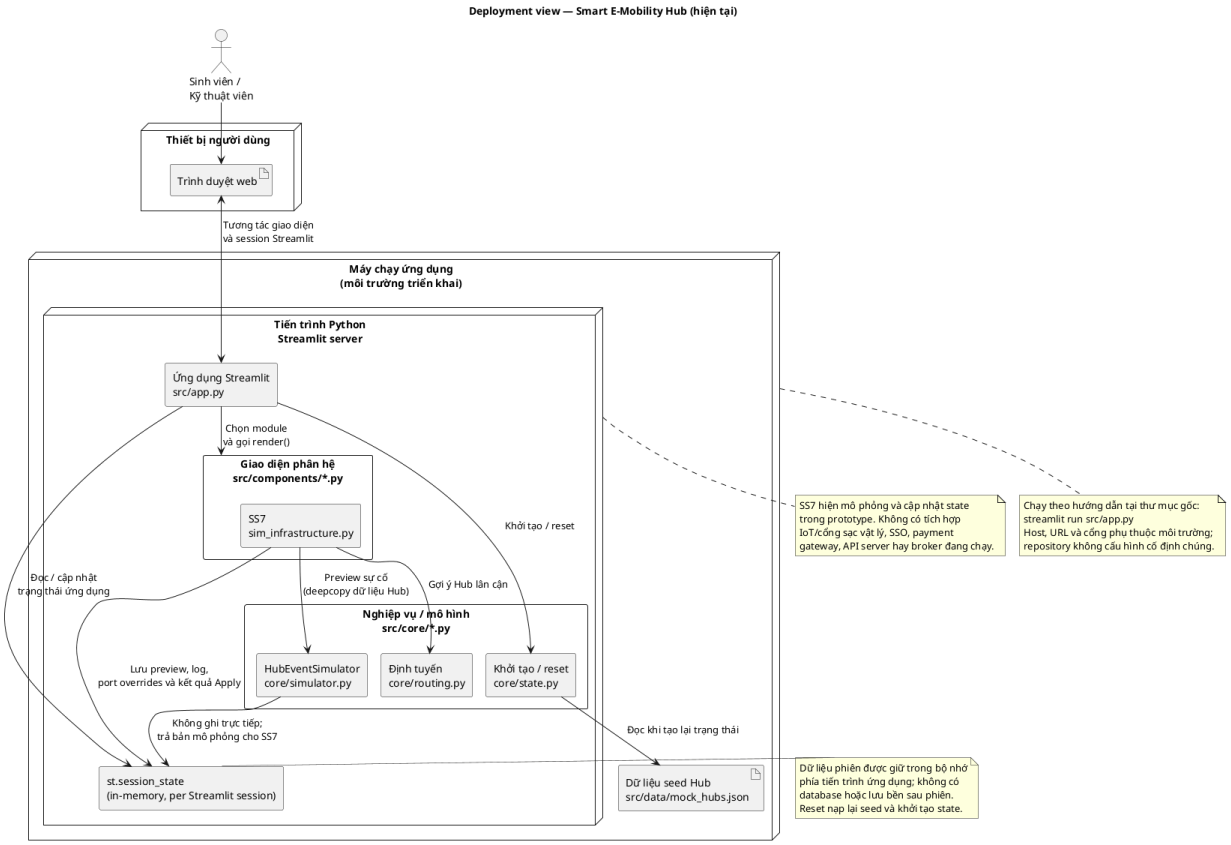


Figure 4: Deployment

PHỤ LỤC: Kiến trúc Hệ thống Chung (architecture.md)

5 Kiến trúc chung revision 06/11/2026

5.1 Deployment view hiện tại

Browser giao tiếp ứng dụng Streamlit/Python; server đọc seed từ src/data/mock_hubs.json khi khởi tạo/reset. st.session_state giữ hubs, ví/ledger, booking, scheduler, preview và version trong bộ nhớ server theo session. Rerun chạy lại script theo tương tác; không có một tiến trình IoT/polling/background riêng. Đổi tab không tự bảo đảm xóa preview. Session/reset/restart không là lưu bền; hai session không phải cùng một cơ sở dữ liệu mạng lưới.

Chạy `streamlit run src/app.py` từ repository. 8501 chỉ là giá trị mặc định công cụ nếu không bị cấu hình đổi, không phải endpoint cố định được triển khai bởi nhóm. SSO, QR/barie/IoT/payment thật là mô phỏng hoặc ngoài phạm vi prototype.

5.2 Development/Implementation view hiện tại

app.py điều hướng tới bảy components; core.booking/personal_ev/billing/scheduler/simulator/rou
chứa phần nghiệp vụ đã tách. SS3/SS4 và Apply SS6/SS7 vẫn có xử lý trong render(). Các service OOP trong class diagram là thiết kế mục tiêu, không là import/package đang chạy. Thư viện Streamlit/Pandas/Plotly phụ trách UI/biểu đồ, không thay lớp miền nghiệp vụ.

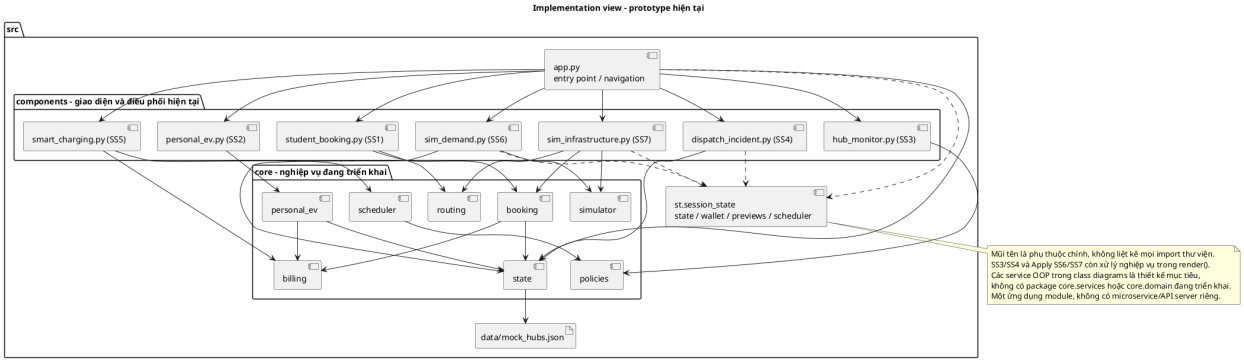


Figure 5: Implementation

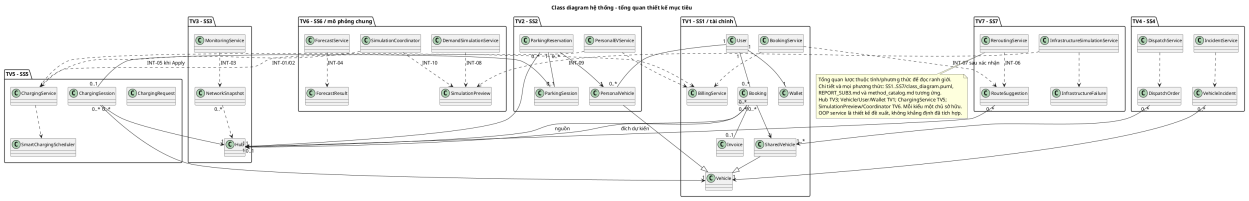


Figure 6: System classes

5.3 Class diagram hệ thống - thiết kế mục tiêu

Sơ đồ tổng quan chỉ thể hiện quyền sở hữu và liên kết; từng SS có sơ đồ chi tiết, lớp/thuộc tính và catalog cho mọi phương thức. Không vẽ lược để né mô tả phương thức. Hub chuẩn TV3, Vehicle/User/tài chính TV1, sạc TV5, preview/coordinator TV6. INT01–10 trong shared_contracts.md là chữ ký thống nhất.

5.4 Ràng buộc chất lượng và hướng mở rộng

Kiểm tra capacity, quyền và version phải nằm tại ranh giới service, không chỉ ở UI. Commit thay đổi Hub/booking/wallet/charging phải nguyên tử trong thiết kế; cùng operation_key/charge_id không được gây tác động hai lần. Để hỗ trợ nhiều người dùng cần storage/version/transaction dùng chung và authentication/authorization thật. SRS về độ trễ, tải và availability cần kế hoạch benchmark/deploy riêng, chưa được chứng minh bởi unit test prototype. Đề giảng viên không bắt buộc database cho Sub3.